ColdRoute SG

Weather-, traffic- and parking-aware route planning
for cold-chain deliveries in Singapore

CS5224 Cloud Computing — Group Project Final Report

Group 36

Agarwal Ishan ů Chao Chien Hung ů Huang Xuan Kun ů Kim Jae Hyeok ů Ler Lian Ping ů Passion Goh Wei Ling

Live service: `https://main.d1f8mppryelp5v.amplifyapp.com/` Code: `https://github.com/legolas456/ColdRouteSG`

Abstract. Small Singapore food and healthcare distributors plan chilled and frozen delivery runs by hand or with general routing tools that minimise driving time. ColdRoute SG is a browser-based Software-as-a-Service console that sequences one vehicle's 5–15 stops using delivery windows, LTA road speeds, NEA rain forecasts, carpark availability and cargo sensitivity, and explains each trade-off against the shortest route. Signed-in plans on live data are computed by a Lambda function with drive times from Amazon Location Service, on a serverless AWS stack (Amplify, API Gateway, Lambda, DynamoDB, S3, Cognito, EventBridge) defined entirely in Terraform. A 50-account pilot would cost about S$132 per month, mostly routing fees, against roughly S$2,135 per month for an equivalent redundant on-premise deployment. On 60 generated scenarios, compared with a planner that only meets booked slots, it carries 18% less cargo time on board for a median 1–7% more driving, and unloads 40% less in rain. On the deployed stack a signed-in plan returns in about 0.35 s when its drive times are cached. **[one clause on the user study.]**

Contents

1 Motivation and Business Case **1**
2 Business Model and Cloud vs On-Premise Justification **2**
3 Architecture and Implementation **3**
4 Evaluation **6**
5 Conclusion and Future Work **7**
6 Code Explanation **8**
Declaration of AI Use **8**

Submitted 13 November 2026. DRAFT v1, 4 Nov 2026: coloured boxes are team notes and disappear in `final.tex`.

1 Motivation and Business Case

Problem. Singapore imports more than 90% of its food [1], and licensed food establishments must keep products at their required temperatures [2]. The last mile is where cold chains are hardest to control: a van makes many short stops, waits for loading space and opens its doors in tropical heat and frequent rain. A dispatcher must order the stops so that every delivery window is met, while deciding how much extra driving is worth paying to avoid unloading in a storm, at a stop with no parking nearby, or keeping vaccines on board for an extra hour. General route optimisers answer a simpler question: *what is the shortest feasible tour?*

Target users. Our buyer is a small Singapore food, pharmaceutical or healthcare-supply operator that runs its own vans. The user is its dispatcher or operations coordinator, who plans one known run from one depot to roughly 5–15 stops before the driver leaves. Receiving restaurants and clinics are not buyers. ColdRoute is a planning aid; it does not navigate, track drivers or measure cargo temperature.

Purpose of the service. ColdRoute (i) fuses Singapore open data that a small operator would never integrate by itself—LTA DataMall speed bands and carparks, NEA two-hour forecasts, HDB carpark availability [3, 4]—with road travel times; (ii) searches stop orders with a transparent, normalised objective whose weights reflect the cargo (Pharma, Balanced, Fastest); and (iii) shows the recommended route side by side with the shortest-drive route, with plain-language reasons for every change. Runs are saved per dispatcher account so that teams can reopen and replan them.

Comparison with existing services. Several mature SaaS products already plan delivery routes (Table 1). They offer full fleet operations—driver apps, tracking, proof of delivery—that ColdRoute does not try to reproduce. Their public pages, however, do not show the three things ColdRoute is built around: Singapore-specific weather and parking feeds, a cargo-sensitivity term, and an explicit explanation of what the chosen order costs and saves compared with the shortest route. We do not claim that competitors *cannot* do these things; where a vendor page is silent we mark the capability as unknown. ColdRoute's differentiation is a narrow, explainable, locally informed planner priced for a two- or three-van operator.

Table 1: Named route-planning SaaS products, from official vendor pages accessed 4 Nov 2026 [5]. Prices use each vendor's own unit and currency symbol and are not directly comparable. "?" = not stated on the vendor's pages.

Product	Public price (billing unit)	Windows	Traffic	Weather	SG feeds	Notes
OptimoRoute	Lite $35.10, Pro $44.10 per driver/month (annual)	Yes	Claimed	?	?	Refrigerated-vehicle matching; analytics
Routific	Free ≤100 orders; $150/month to 1,000 orders	Yes	Historical	?	?	Unlimited drivers
Onfleet	Launch from $619/month (2,500 tasks)	Yes	Claimed	Blog	?	Full delivery management, POD
Spoke Dispatch	US$125/month (1,000 stops)	Yes	Navigation	?	?	Formerly Circuit for Teams
ColdRoute SG	**S$49 per active vehicle/month** (proposed)	**Yes**	**LTA live/typical**	**NEA 2-h**	**LTA, NEA, HDB**	**Cargo term; explains trade-off vs shortest**

Assumptions. (A1) One vehicle per plan, departing from one depot, with no return-to-depot cost in the objective. (A2) Service (unloading) time per stop is entered by the dispatcher and does not change with weather or parking. (A3) Carpark availability within 600 m and the nearest NEA forecast area are reasonable proxies for unloading conditions; an available car lot is not proof of a legal loading bay. (A4) "Cargo exposure" is sensitivity-weighted time on board, not a temperature measurement. (A5) All prices exclude GST; vendor prices are shown as published. (A6) Cloud costs are modelled from AWS price lists and our measurements, not taken from a bill; user-study participants were mostly students, not dispatchers.

Response to preliminary feedback. Table 2 lists where each of the TA's five comments is answered.

Table 2: The TA's comments on our preliminary report and where this report answers them.

Comment	Answer
Name competitors; make the difference precise	Table 1: four named products, with prices and what each page does not show
Give the business model numbers	S$49 per active vehicle, 30-day pilot; market ceilings (Table 3); cloud and on-premise costs (Section 2)
Set measurable acceptance criteria	Targets fixed before evaluation for route quality, usability and cost (Section 4)
Explain how data at different scales is combined	Leg factors and stop conditions on one frozen snapshot (Section 3.4, Table 6)
Normalise the terms; justify the weights	Fixed per-stop reference scales, profile weights and sensitivity test (Section 3.5, 4.2)

2 Business Model and Cloud vs On-Premise Justification

Revenue model. The course prototype is free. For a commercial version we propose one subscription, **S$49 per active vehicle per month** after a 30-day free pilot, including the dispatcher account, saved runs and live data. A vehicle is active in a month if at least one run was planned for it. Pricing per vehicle matches the product, which plans one van at a time, and an idle van costs the customer nothing, as it costs us almost nothing. S$49 is about US$38 [9], between OptimoRoute's Lite and Pro per-driver prices [5]. A three-van operator pays S$147 (about US$115), less than Routific's US$150 tier for its roughly 660 orders a month (10 stops a van, 22 days) and far less than Onfleet's entry tier. ColdRoute manages less of the fleet than these tools, so the price assumes its cold-chain planning is worth as much to these buyers; the pilot would test that. It is a hypothesis, not a measured willingness to pay.

Market size. Enterprise Singapore's Logistics Industry Digital Plan targets about 5,300 logistics SMEs [6]. This is a broad, dated denominator, not a count of cold-chain operators, so we apply explicit eligibility filters (Table 3): annual ceiling = 5,300 × eligible share × vans per firm × S49 × 12. These figures assume every eligible firm subscribes; they are upper bounds, not revenue forecasts.

Table 3: Illustrative addressable subscription revenue.

Scenario	Firms	Vans	Annual ceiling
Low (2%)	106	1	S$62,328
Base (5%)	265	2	S$311,640
High (10%)	530	3	S$934,920

Table 4: Modelled monthly AWS cost (USD, no free tier).

Line item	Demo	Pilot (50)	Growth (500)
Lambda (4 functions)	0.56	1.74	12.21
Location Service routes	0.30	90.00	900.00
API Gateway (HTTP)	<0.01	0.48	4.85
DynamoDB (history + cache)	0.01	3.30	33.05
S3 + Amplify hosting	0.22	1.45	12.61
Cognito (Essentials)	0.15	0.75	7.50
CloudWatch	3.01	3.53	8.22
Data transfer out	0.06	2.08	20.23
Total USD	**4.31**	**103.33**	**998.67**
Total S$	**5.51**	**132.01**	**1,275.90**

Cloud cost model. We derived the workload from the Terraform configuration and the team's measurements, then priced it with AWS's official price lists, for Singapore and, for Location Service, Sydney [7] (Table 4). Two loads matter. The *fixed* load is the traffic refresh (1 GB, about 7 s, every 10 minutes: 4,320 runs per month) and the nightly typical-traffic build; it costs under a dollar. The *per-use* load is dominated by routing: Amazon Location Service bills US$0.50 per 1,000 Core routes [8], and a matrix bills every origin–destination cell (a new stop in a run of n points costs about $2n$ routes). Every pair is cached in DynamoDB for 30 days, so replans and recurring customers cost little; on the deployed stack the first plan of the 7-stop sample (8 points) billed 63 routes, a repeat none, and one new stop 17 (its row and column) plus up to 2 for road lines. Drawing the two plans' road lines adds one or two routes per plan unless the planner instance still holds them in memory; the table excludes this, and at worst (two per plan) it adds US$30 (pilot) and US$300 (growth) per month. Scenarios: *demo* has 10 accounts and 5 plans each; *pilot* 50 and *growth* 500 accounts, each making 20 plans per day and meeting about 120 uncached pairs per day (one 10-stop run of entirely new addresses). Each plan is a 1 GB planner invocation of about 1 s. We assume 200 ms API calls, 8 KiB saved plans and 5 MiB of frontend download per account-day, exclude free tiers and convert at US$1 = S$1.2776 [9]. Routing dominates, so the result is most sensitive to the 120 new pairs: at 30 (mostly repeat customers) the pilot's routing falls from US$90 to about US$22.

TO DO 1: The biggest assumption, 120 uncached drive-time pairs per account-day, is about customer behaviour and cannot be measured without customers; keep it labelled as an assumption. What the deployment test can check is the billing arithmetic it rests on: the `/api/plan` response's `roads.billed` (browser DevTools, Network tab) should be $n^2 - 1$ routes for a run of n new points, about $2n$ for one new stop, 0 for a cached repeat, plus 1–2 for road lines. If the account owner can share the actual bill for the test period, one sentence on it is optional; the comparison itself is modelled, as stated.

On-premise equivalent. The same service with comparable availability would need two servers in separately powered sites (active/standby), running the web app, API, scheduled collectors, replicated PostgreSQL and a self-hosted identity provider (Keycloak) instead of Cognito. Using a Dell PowerEdge R350 at its Singapore list price [10], a 36-month write-off, two business fibre links with static IPs [12], SP Group electricity tariffs [13] and 0.1 of a system administrator's time [14] gives about **S$2,135 per month plus S$21,336 up-front** (Table 5). That is a lower bound: it excludes failover testing, a quorum witness and hardware quotes, and two server rooms still do not match DynamoDB's replication across three Availability Zones [15].

Justification. For a start-up with uncertain demand the cloud deployment is clearly better: about 18 times cheaper for the demo and 9 times cheaper for the pilot, with no up-front capital, near-zero idle cost, automatic scaling and no servers to patch. Its replicated storage and managed authentication would be costly to reproduce on premises. Even the cheapest option we could find, one server in a colocation rack (from S$170/month [11], before power and bandwidth quotes) with the same write-off, backups and 0.1 FTE, costs about S$1,040/month: four times the pilot, with no failover. The margin narrows with scale because routing is a per-use fee: at 500 accounts the cloud

Table 5: Cloud vs on-premise, monthly S$ (excl. GST). Cloud includes 0.02 FTE of operations time; on-premise 0.10 FTE.

On-premise item	S$/month	Comparison	S$/month
2 servers + UPS/network, 36-month write-off	592.66	AWS demo (incl. labour)	118.07
Rack space, 2 sites (assumed)	300.00	AWS pilot, 50 accounts	245.38
Power and cooling (250 W, ×1.5)	77.19	AWS growth, 500 accounts	1,396.56
2 business internet links, static IP	572.48	On-premise (any scale here)	2,135.13
Backups + 0.1 FTE sysadmin	592.80	Up-front on-premise capex	21,335.78
Total, redundant on-premise	**2,135.13**	Minimal: 1 colocated server, no failover	1,038.30

is 1.5 times cheaper than the redundant design but dearer than a single server, and a linear extrapolation puts break-even with the redundant design between about 600 accounts (road lines at their worst) and 800, above our high market scenario (Table 3). The on-premise design avoids this fee by self-hosting OSRM on free OpenStreetMap data; the cloud design can do the same (OSRM on a container service) or cache per depot if routing ever dominates. At S$49 per vehicle, pilot infrastructure costs about S$2.64 per account-month. On-premise would give physical control of data, but PDPA obligations apply either way and choosing a region does not by itself make the service compliant [16]. The trade-off is lock-in to Cognito, DynamoDB, Amplify and Location Service interfaces.

3 Architecture and Implementation

3.1 Requirements

Functional. F1 enter a depot, departure time, cargo profile and up to 15 stops (OneMap search) with windows, unloading minutes and cargo type; F2 plan a route with recorded or live conditions; F3 compare with the shortest route and explain differences; F4 sign in, save, reopen and delete runs (dispatcher) and view team activity (admin); F5 export a plan as CSV. **Non-functional.** Plans in about a second for 15 stops; usable when any live feed fails (recorded/typical fallback); no secrets in the browser; each user can read only their own runs; infrastructure reproducible from code; near-zero idle cost.

3.2 Cloud architecture

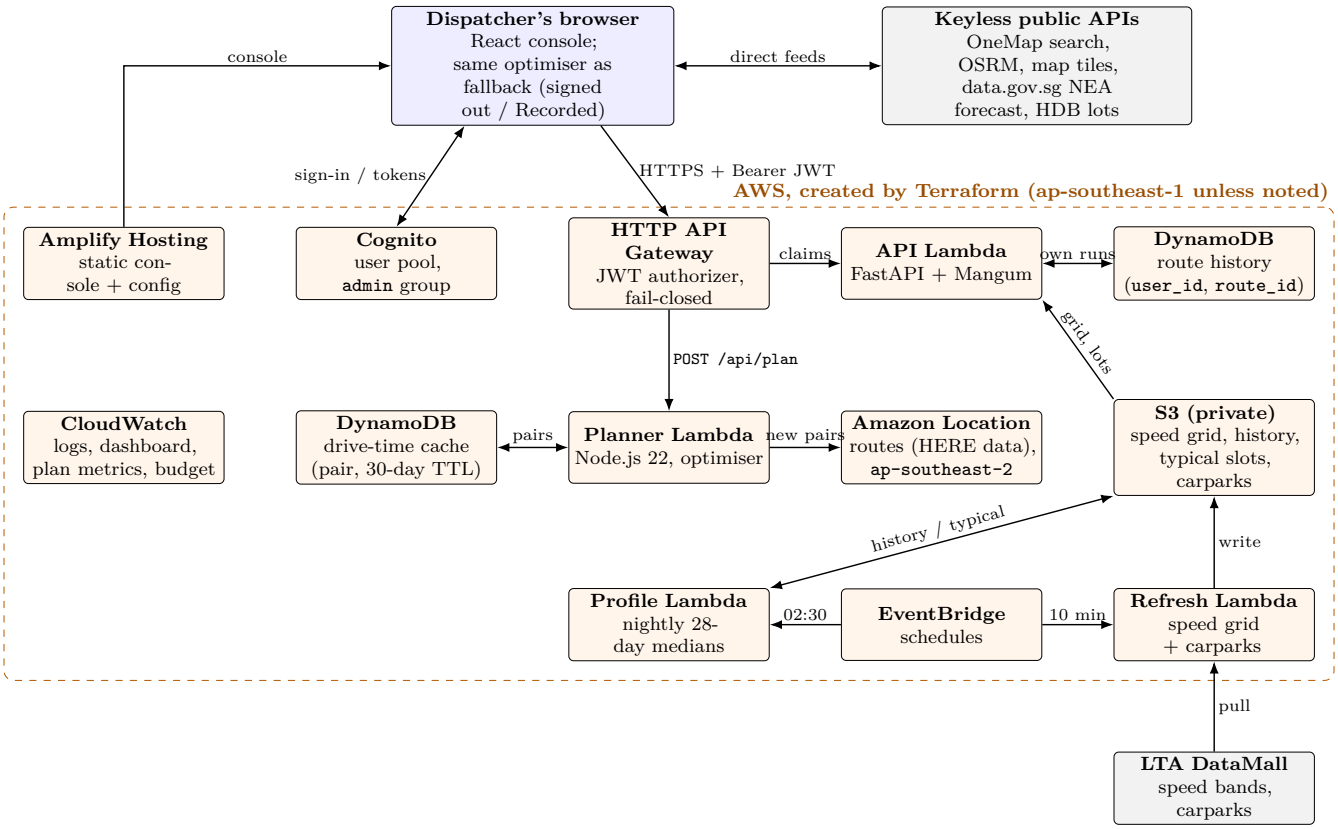


Figure 1: Deployed architecture. A signed-in plan on Live data runs on the planner Lambda, which reads the S3 snapshot and data.gov.sg and asks Location Service only for drive-time pairs it has not cached. One shared refresh keeps the LTA key and LTA call volume independent of the number of users.

The whole stack (Figure 1) is declared in Terraform: one `terraform apply` creates every resource, packages the Python Lambda ZIP and the bundled Node.js planner, builds the React console with the API and Cognito IDs injected, and uploads it to Amplify. *API*: an HTTP API, throttled at 50 requests/s (burst 100), routes `/api/plan`

to the planner and everything else to a FastAPI application adapted to Lambda by Mangum (Python 3.12, 512 MB). *Planner*: a Node.js 22 Lambda (1 GB, 29 s) that bundles the console's own TypeScript conditions and optimiser, so the same drive times and conditions give the same plan wherever it is made. It reads the NEA forecast and HDB lots, the S3 traffic snapshot and typical slots, then builds the drive-time matrix from cached pairs plus new pairs from Amazon Location Service. Location Service runs in Sydney (`ap-southeast-2`, HERE data, same price), because the Singapore region's map data could not route to or from one test hospital (NUH). It asks for free-flow times, and LTA traffic is applied on top as in the browser. If it fails, the planner falls back to OSRM and then to a distance estimate, and the console falls back to planning in the browser, always saying which source was used. *Data pipeline*: every 10 minutes EventBridge triggers the refresh Lambda (1 GB). It pages through LTA's speed bands, aggregates them into a 0.01° (≈1.1 km) delay-factor grid, stores the latest grid and a timestamped copy in S3, then refreshes URA and mall carpark counts. Each night a profile Lambda reads the last 28 days of grids and publishes per-cell *median* factors for each weekday/weekend 15-minute slot, used when a departure is more than an hour away from the live reading. *Persistence*: DynamoDB (on-demand) stores saved runs under partition key `user_id` (the Cognito `sub`) and sort key `route_id` (timestamp#UUID). *Operations*: logs are kept 14 days; a CloudWatch dashboard shows API p50/p95 latency, errors, and each plan's time and routes billed; an optional AWS Budget emails at 80% of US$10. This stack replaced our first prototype, a hand-deployed Lambda, Cognito and DynamoDB backend with a greedy optimiser; its saved-run and admin code was carried over, and its `/api/routes/optimize` endpoint is still deployed but no longer used by the console.

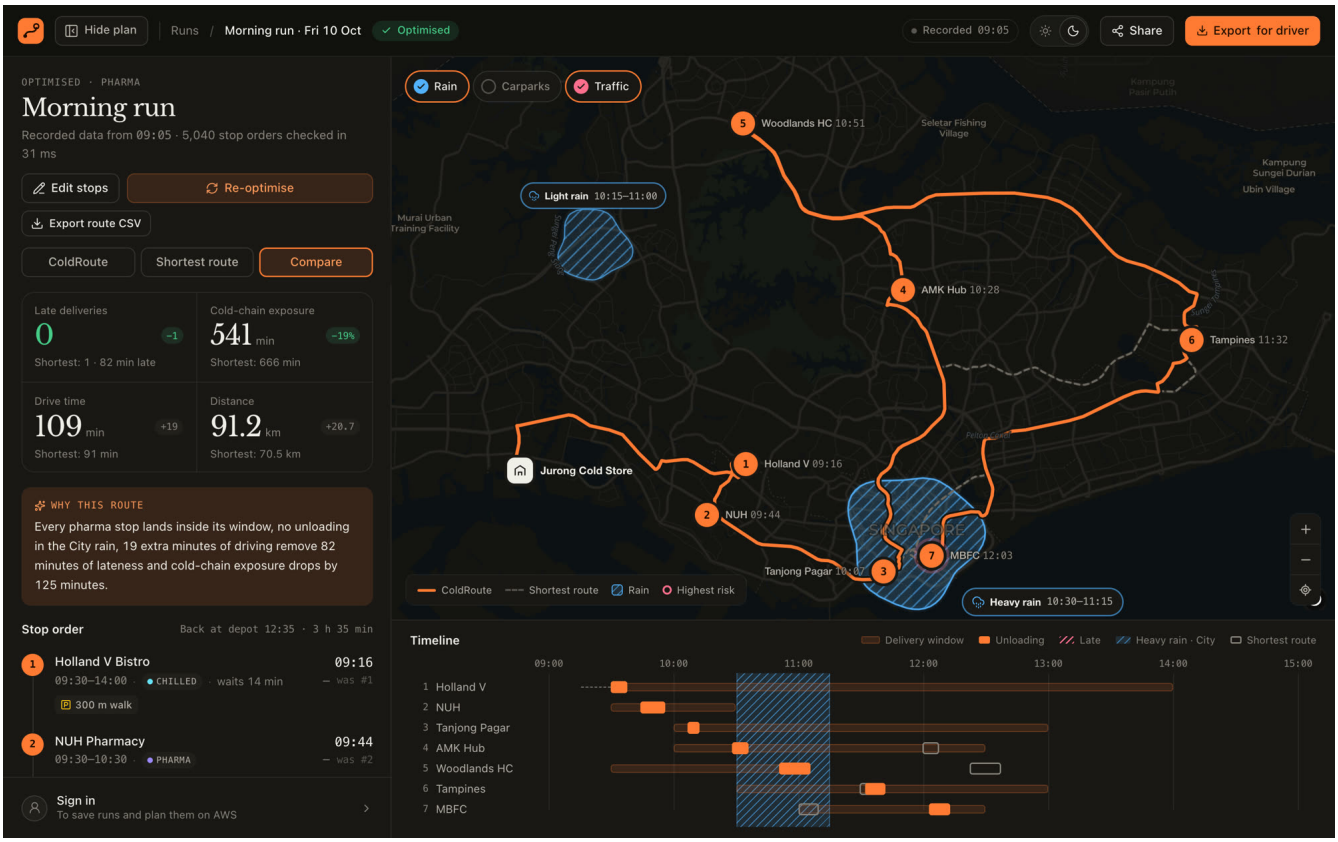


Figure 2: The deployed console in *Compare* view on the recorded sample (Pharma, signed out, so planned in the browser): ColdRoute's order (solid) against the shortest route (dashed), the rain cells it plans around, the trade-off metrics and the stop timeline. Here ColdRoute has no late stop (shortest: one, 82 min late) and 19% less cargo exposure for 19 min more driving.

3.3 User journey

1. The dispatcher opens the console (no install). It works immediately in *Recorded* mode with sample Singapore data; signing in (Cognito, administrator-issued accounts) enables saved runs.
2. They choose the depot, departure time and profile, then add stops via OneMap address search, editing each stop's window, unloading minutes and cargo (Pharma, Frozen, Chilled).
3. They switch to *Live* to see current rain areas, carpark counts and slow roads. Every source shows its timestamp.
4. *Optimise* freezes the current conditions and sends the run to the planner Lambda (signed in, Live), which returns both the recommended and the shortest-drive order on the same inputs; otherwise the browser plans it.
5. *Compare* shows late stops, drive time, exposure and changed positions, while each stop card explains its placement. The plan is saved to *Your runs* and can be exported as CSV.

3.4 Combining data with different granularity

The TA asked how signals measured on different spatial and temporal scales are combined. We never mix raw feeds. Each source keeps its native key and timestamp, and is converted into one of two derived inputs: a *leg factor* that scales an OSRM travel time, or a *stop condition* evaluated at the simulated unloading time (Table 6). Conditions are frozen when the user presses Optimise, so both routes are scored on an identical snapshot.

Table 6: How each signal is aligned to legs and stops (as implemented).

Signal	Native granularity	Spatial / temporal alignment	Freshness and fallback
Road travel time	point-to-point matrix	base minutes t^0_{ij} per ordered pair from Amazon Location Service (cloud) or OSRM (browser)	pairs cached 30 days; OSRM, then a 1.35× distance at 38 km/h estimate if routing fails
LTA speed bands	road links, 5-min updates	length-weighted night/current speed ratio per 1.1 km cell (clipped 1–3); a leg averages cells at points every 0.5 km	live grid if ≤20 min old and departure within ±60 min, else typical median for that slot, else factor 1
NEA 2-h forecast	47 named areas, 2-h validity	stop takes the nearest area; rain counts if the unloading interval overlaps the forecast validity	recorded sample if the feed fails; outside the forecast horizon no rain penalty, with a warning
HDB / URA / mall carparks	carpark, ∼1-min updates	nearest reporting carpark within 600 m; risk from walk distance and free share	URA/mall counts older than 30 min dropped; otherwise the stop keeps its recorded estimate
Traffic cameras	camera points, images	displayed near stops only; never used as numbers	–

TO DO 2: Known data-fusion limits to either fix before submission or keep in Section 5: (a) the traffic factor samples the straight line between stops, not the routed road; (b) one traffic snapshot is used for the whole run regardless of when each leg is driven; (c) weather beyond the forecast horizon is treated as dry rather than unknown; (d) parking availability is fixed for the run, not predicted for the arrival time.

3.5 Route objective, normalisation and search

For a stop order r, the simulator computes arrival A_k, start of service $S_k = \max(A_k, a_{r_k})$ (waiting if early) and finish $F_k = S_k + s_{r_k}$, with leg times $t_k = t^0 f$ scaled by the traffic factor f. It then evaluates five raw terms: drive time $T = \sum t_k$; lateness $L = \sum \max(0, S_k - b_{r_k})$; weather exposure $W = \sum q_k s_{r_k}$, where $q_k = 1$ for heavy or thundery rain and 0.5 for other rain overlapping the service; parking difficulty $P = \sum p_{r_k} s_{r_k}$, with $p = \min(1,\, 0.6\min(d/1.2\,\text{km}, 1) + 0.4(1 - \text{free}\%))$; and cargo exposure $C = \sum h_{r_k}(S_k - t_0)$ for departure time t_0, where the sensitivity h is 1.0 for Pharma, 0.8 for Frozen and 0.6 for Chilled. Following the TA's advice, each term is divided by a fixed, per-stop reference scale, so the weights are unit-free and comparable:

$$J(r) = \alpha\frac{T}{20n} + \beta\frac{L}{15n} + \gamma\frac{W}{15n} + \delta\frac{P}{15n} + \varepsilon\frac{C}{120n}, \qquad \alpha+\beta+\gamma+\delta+\varepsilon = 1. \tag{1}$$

The scales are fixed rather than rescaled per candidate, because min–max scaling would change the meaning of a weight from one run to the next. Lateness is not capped. The weights are profiles chosen by the dispatcher: *Pharma* $(0.15, 0.35, 0.10, 0.05, 0.35)$ protects windows and on-board time; *Balanced* $(0.30, 0.30, 0.10, 0.10, 0.20)$ trades a little driving for lower risk; *Fastest* $(1, 0, 0, 0, 0)$ reproduces shortest-drive routing. They are pilot hypotheses chosen by the team, not values fitted to data. One term needs care: P is the same for every order of a given run (a fixed risk times a fixed unloading time), so parking shifts the score but cannot change the route. We keep it as a per-stop risk flag; making it order-dependent would need parking predicted for each arrival time, which the feeds do not give. Section 4.2 tests how much the result depends on them: moving any one weight by ±20% changes few routes and moves exposure and driving by about 1%; lateness is the most sensitive.

Search. With up to 8 stops ($8! = 40{,}320$ orders) the optimiser enumerates every permutation and is exact. Above 8 it runs deterministic local search from four starts (shortest-drive, entered order, earliest deadline, nearest neighbour), applying Or-opt moves of 1–3 stops, 2-opt reversals and swaps until no improvement. The shortest-drive baseline is solved exactly up to 15 stops with Held–Karp dynamic programming [17]. Starting the search from the baseline guarantees that the recommendation scores no worse than it. The same code runs on the planner Lambda and in the browser; only the drive times differ (Location Service in the cloud, OSRM in the browser), so the same run can be ordered differently in the two places.

3.6 Security, privacy and scalability

The API *fails closed*: only health, public data, traffic grids, carparks and CORS preflight are public, and every other route, including `POST /api/plan`, requires a valid Cognito JWT checked by API Gateway (an unsigned plan request gets 401). The backend takes the user's identity only from the verified `sub` claim, and each DynamoDB query, read or delete is restricted to that user's partition, so guessing another run's ID returns nothing. Admin endpoints also check the `admin` group server-side. Each Lambda's IAM role allows only what it uses. The LTA key exists only in the refresh Lambda's environment and never reaches the browser; S3 blocks all public access. Saved runs, dispatcher e-mails and the drive-time cache stay in `ap-southeast-1`. Only stop coordinates, never names or

addresses, leave Singapore: to Location Service in Sydney and, if it fails, to the public OSRM server. Most stops are businesses, but a private home's coordinates can be personal data, so a commercial version would meet PDPA's Transfer Limitation Obligation [16] through the provider's data-processing terms, or route inside Singapore with self-hosted OSRM. Scalability comes from the design: planner and API Lambdas scale per request, DynamoDB is on-demand, the drive-time cache makes repeat plans free, and one shared refresh serves all users, so LTA usage stays constant (≈4,320 runs/month) however many users there are. Because each plan with new stops costs routing fees, `POST /api/plan` has its own throttle (5 requests/s for all users together), below the API's 50/s.

TO DO 3: Security/scalability caveats to decide on: the LTA key also sits in Terraform state (move to Secrets Manager/SSM?); the admin analytics endpoint scans the whole table (fine for the course, not for 500 accounts).

4 Evaluation

We evaluated four questions: does the system behave correctly, does the recommendation actually trade driving for lower risk, is it fast and reliable on AWS, and can people use it.

4.1 Correctness: automated tests and deployment checks

The repository has 340 automated tests, all passing on the final code: 135 Vitest and 12 Node tests for the console, 24 for the planner (with fake Location Service, DynamoDB and S3 clients) and 169 pytest tests for the backend. GitHub Actions runs them, the TypeScript builds, the planner bundle and `terraform validate` on every pull request. They check, among other things, that every stop appears exactly once; that waiting, lateness and rain overlap are simulated correctly; that the heuristic matches exhaustive search on fixed 9-stop seeds and is repeatable at 12 stops; the traffic-grid maths, typical-traffic slot medians and stale-carpark rejection; JWT ownership of saved runs; that the planner never re-measures a cached pair; and that the recorded demo route stays fixed. After deployment we ran an end-to-end checklist on the live stack (Table 7).

Table 7: Deployment acceptance checks on the live AWS stack (5 November 2026).

Check	Result	Check	Result
Console loads from Amplify URL over HTTPS	Pass	Signed-out requests to 6 protected routes (incl. `/api/plan`) return 401	Pass
Sign-in and first-login password change (3 accounts)	Pass	User B reading user A's run gets 404; B's delete of it is accepted (204) but removes nothing	Pass
Optimise → run saved, listed, reopened, deleted	Pass	Admin sees team activity; dispatchers get 403	Pass
Speed grid in S3 refreshed every 10 min (Live traffic)	Pass	Typical slots published at 02:30 (e.g. weekday 09:00: 10 readings, 5 days)	Pass

4.2 Route quality

Protocol. We generated 60 scenarios: five types (an *easy* control with no booked slots or rain; *tight* 30–60 min booked slots at half the stops; heavy *rain* over some stops' areas; *mixed* Pharma, Frozen and Chilled cargo, all at least 8 km from the depot; and all *combined*), each with 5, 10 and 15 stops and four seeds. Stops come from 43 real Singapore addresses: the ten sample customers, plus one HDB block per mainland NEA forecast area standing in for a customer there. Every run leaves the Jurong depot at 09:00, with drive times from one saved OSRM matrix and no traffic. Slots sit on the arrival times of a random order, which the script checks meets every slot, and rain starts at a random time in the first half of the run. Each scenario is planned five ways with the console's own code: the exact *shortest* drive; earliest deadline first (*EDF*, equal deadlines nearest first), the order a careful dispatcher might pick by hand; *slots-only*, our optimiser with only its drive and lateness terms, standing in for a general route planner that respects time windows; and ColdRoute's *Balanced* and *Pharma* profiles. The weights were set before the run and not tuned, so all 60 runs are reported. Lateness, rain and exposure are terms of J, so we report raw minutes, and slots-only is the main comparison: it shows what the rain and cargo terms add beyond time windows. The script, saved drive times and per-run results are in `frontend/eval` (`npm run eval`, about 30 s).

Table 8: Route quality. "With slots" is the 24 *tight* and *combined* runs; rain, mixed and easy are 12 runs each. Late figures and rain-exposed unloading are totals. Exposure is the change in total sensitivity-weighted time on board versus slots-only; driving is the median change versus slots-only; easy shows the worst extra driving versus the shortest route.

	24 runs with slots				Rain	Mixed	Easy
Plan	Late stops	Late min	Exposure	Driving	rain min	exposure	worst extra
Shortest drive	97	16,578	+65.7%	−18.9%	88.7	0%	0%
EDF	12	501	+41.0%	+20.4%	90.0	−3.1%	23.4%
Slots-only	2	12	0%	0%	88.7	0%	0%
Balanced	10	56	**−18.0%**	+1.1%	**53.0**	−4.4%	4.1%
Pharma	4	9	**−18.4%**	+6.8%	**59.0**	−6.6%	11.5%

Results. The three targets set before the run are met on the 48 constrained runs: against the shortest route,

Balanced cuts late minutes by 99.7% and cargo exposure by 36.5% for a median 5.3% more driving (Pharma: 99.9%, 37.1%, 10.7%). These figures flatter us, because the shortest route ignores slots: it misses 97 of them by 171 min on average, and that lateness also counts as time on board. Against slots-only, which meets the slots too, the gain is smaller but consistent: on the 24 runs with slots Balanced carries 18% less cargo exposure for a median 1% more driving, better in 17 runs and worse in 1 (sign test $p < 0.001$); Pharma cuts 18% for 7% more driving (20 better, 1 worse). The price is a little lateness: Balanced is late 56 min in total against slots-only's 12, because its cargo term accepts a few late minutes to shorten time on board. Meeting slots is what costs driving: on runs with slots, Balanced drives a median 26% more than the shortest route (IQR 14–33%) and slots-only 23%. In the rain runs ColdRoute unloads 40% less in rain for no extra median driving (better in 4 runs, never worse). On the easy control Balanced kept the shortest route's driving in 9 of 12 runs. **Where it helps least.** With mixed cargo but no slots, most orders keep cargo on board for similar times, so exposure falls only 4–7%, below the 10% target for that type. With slots and rain together, both profiles still unload in rain for 67–79 min (slots-only: 82), because they weight a late minute about three times a wet one.

Ablation and sensitivity. Switching off one Balanced term (the rest renormalised, 48 constrained runs): without weather, 6 routes change and rain unloading rises from 120 to 182 min; without cargo, 27 change, exposure rises 13.5% and late minutes fall from 56 to 13; without lateness, late minutes reach 2,263; without parking nothing changes (Section 3.5). Moving any one weight by ±20% changed at most 11 of the 60 routes and total exposure or driving by at most 1.3%, but late minutes by up to 38 min on a total of 56, so lateness is the result most sensitive to the weights.

4.3 Performance and reliability

Search quality and speed. Against exhaustive search on 80 random 9-stop runs (Balanced and Pharma), the heuristic found the optimum in 78, was at worst 3.4% above it and never had more late stops; it was optimal on all 10 random 10-stop runs. A plain 2-opt from the entered order found only 29 of the 80 (worst 47% above). On the 60 evaluation scenarios we also compared it with an exact branch-and-bound search: it found the optimum in 39 of 40 ten-stop plans (worst 1.75% above, mean 0.04%). Median planning time on a laptop (Node 22) is 0.6 ms for 5 stops, 9 ms for 10 and 54 ms for 15 (always under 100 ms). **Cloud latency.** On the production stack (3 Nov), the first plan of the 7-stop sample took 3.7 s including a Lambda cold start and 63 new routes. On 5 November 2026, timed end to end from a laptop in Singapore with a signed-in test account, 18 repeat plans of the cached sample over two runs took a median of 0.35 s (0.29 s inside the planner; at most 1.2 s). Plans adding one or two stops new to the cache took 1.0–2.7 s for 16–38 new pairs (17–40 billed routes); repeating such a plan took 0.26 s and billed nothing. Billing matched the cost model: one new stop on the 8-point sample is $2 \times 8 = 16$ new pairs, billed as 17 matrix routes plus up to 2 for road lines. Read every 5 minutes over 2 hours (24 reads, none failed), our copy of the road-speed grid was at most 10 minutes old and LTA's own timestamp at most 18 minutes. The console judges staleness by LTA's timestamp and falls back to typical traffic past 20 minutes, so in this window it never did, though with only about 2 minutes to spare.

4.4 User evaluation

Method. [N] participants used the deployed console signed out, in Recorded mode, without help, and then answered an anonymous online form (about 10 minutes). They did five tasks on the sample run: switch to Balanced and back to Pharma, change Tampines Mall Cold Store's closing time to 11:00, optimise, compare with the shortest route, and export the CSV. Because every participant saw the same run, the first stop of their plan has one correct answer. We measured self-reported task completion, that answer, whether they could explain in their own words why the route differed from the shortest one (an answer counts if it names rain, a time window or lateness, or cargo exposure; two members coded each answer separately), and usability with the standard 10-item System Usability Scale (SUS) [18]. A SUS score of 68 is commonly treated as average [19]. Our targets were 90% task completion, 80% correct explanations and SUS ≥70.

Results. **[completion per task; % correct first stop; % correct explanations and coder agreement; mean SUS ± SD; 2–3 representative comments and what we changed.]**

DECIDE 4: Report collected responses on their own, with their count. Any figure that is not from collected responses must be labelled "for illustrative purposes" next to it, with its count. State that participants were mostly students, so this tests the interface's usability, not the market. Analyse with `analyse_form.py` (see the user-study kit).

5 Conclusion and Future Work

ColdRoute SG shows that a small team can turn Singapore's open transport and weather data into a usable, explainable planning service on a serverless stack. Its fixed hosting cost is a few dollars a month, it is roughly nine times cheaper than an equivalent on-premise deployment at pilot scale, signed-in plans return in about 0.35 s from cached drive times, and the whole stack can be rebuilt with one `terraform apply`. On 60 test scenarios it meets nearly every booked slot and, compared with a planner that only respects slots, carries 18% less cargo time on board for a median 1–7% more driving; it helps least when cargo mix is the only difference. **[one sentence on the user study.]** **Limitations.** The objective uses proxies (sensitivity-weighted time on board, parking within

600 m, nearest forecast area), not measured temperature or loading-bay access. The weights are expert-set. Traffic is a per-run snapshot sampled along the straight line between stops, and the parking term cannot change the order. Plans cover one vehicle without a return leg, and our users were not professional dispatchers. **Future work.** Multi-vehicle planning; a driver app with live re-optimisation; reefer temperature telemetry to replace the exposure proxy; time-dependent traffic per leg; weights calibrated from operators' choices; and a pilot with real operators to test the S$49 price.

6 Code Explanation

Repository: `https://github.com/legolas456/ColdRouteSG` (public until marks are released). **Live service:** `https://main.d1f8mppryelp5v.amplifyapp.com/`. Test login for graders: **[e-mail / password, or "on request"]**.

TO DO 5: Make the repository public (agreed) and check the link works when logged out. Delete merged branches first.

Path	Contents
`frontend/src/lib/`	optimiser (`optimizer.ts`, `objective.ts`, `profiles.ts`), data fusion (`conditions.ts`, `traffic.ts`, `live.ts`, `feeds.ts`), explanations, Cognito auth, saved runs, planner client
`frontend/src/components/`	React UI: stops editor, map (MapLibre), results/compare, timeline, conditions panel, account and saved runs
`planner/`	cloud planner Lambda: request validation, inputs from S3/data.gov.sg, Location Service matrix with DynamoDB cache
`backend/app/`	FastAPI app: routers (`routes`, `traffic`, `parking`, `admin`, `data`), services (DynamoDB, JWT claims, LTA grid, typical medians)
`backend/traffic_*.py`	Lambda handlers for the 10-minute refresh and the nightly typical-traffic build
`infra/`	Terraform for every AWS resource, packaging scripts, `README` with deploy steps and cost notes
`frontend/eval/`	route-quality evaluation (Section 4.2): scenario generator, saved drive times, per-run results (`npm run eval`)

Run locally: `make setup && make dev` starts the API on port 8000 and the console on port 5173; `make test` runs the backend and console tests (the planner's are in `planner/`, `npm test`). The console works without the backend in Recorded mode. Live LTA traffic needs a DataMall key in `backend/.env`. **Deploy:** copy `infra/terraform.tfvars.example`, set the LTA key, then run `terraform init && terraform apply`. Terraform prints the console URL, API URL and Cognito IDs; dispatcher accounts are created in Cognito by an administrator.

Declaration of AI Use

[Each member's statement.] Example structure: *Where:* AI coding assistants were used to draft parts of the frontend and Terraform, to review pull requests, to gather and check competitor and AWS pricing, and to draft sections of this report. *How:* every generated change went through a pull request, automated tests and review by another member; every fact and price in this report was checked against the code or the cited source. *What we learned:* **[e.g. how JWT authorizers and partition keys enforce ownership; how fixed reference scales make weights interpretable; reading AWS price lists]**.

DECIDE 6: Each member writes 1–2 lines on what AI they used, where and what they learned (the course requires where, how and learning). Ishan: Manus was used for PR review and fixes (PRs #20–24), fact-checking and this draft; Claude (Claude Code) for an independent review of the code and this draft, re-checking AWS prices against the official price lists, and the form-analysis script.

DECIDE 7: Optional, 2–3 lines if space allows: who did what. Proposed from the chats and PRs, for each member to correct: Kun redesigned the console (PRs #1–9); Lian built the first AWS backend (Cognito, DynamoDB, admin); Chien wrote the normalised objective, depot picker and CSV export, and started the LaTeX report; Jay integrated the planner and built the Terraform stack, live traffic and the cloud planner (#12–27); Ishan wrote the first optimiser tests, reviewed #15–19 with fixes #20–24, ran the user study and drafted this report; Passion drafted the business and early report sections.

References

[1] Singapore Food Agency, "Singapore Food Statistics 2025." `https://www.sfa.gov.sg/news-publications/newsroom/singapore-food-statistics-2025`

[2] Singapore Food Agency, "Responsibilities of food establishment operators." `https://www.sfa.gov.sg/food-manufacturing-storage/operation-requirements-responsibilities/responsibilities-of-food-establishment-operators`

[3] Land Transport Authority, "LTA DataMall API User Guide" (Traffic Speed Bands, Carpark Availability). `https://datamall.lta.gov.sg/content/dam/datamall/datasets/LTA_DataMall_API_User_Guide.pdf`

[4] data.gov.sg, "2-hour Weather Forecast" and "HDB Carpark Availability." `https://api.data.gov.sg/v1/environment/2-hour-weather-forecast`, `https://data.gov.sg/datasets/d_ca933a644e55d34fe21f28b8052fac63/view`

[5] Vendor pricing and feature pages: OptimoRoute `https://optimoroute.com/pricing/`, `https://optimoroute.com/features/`; Routific `https://www.routific.com/pricing`; Onfleet `https://onfleet.com/pricing`; Spoke `https://help.spoke.com/en/articles/1925291-spoke-products-pricing`

[6] Enterprise Singapore, “Annual Report 2020–2021,” p. 27. https://www.enterprisesg.gov.sg/-/media/esg/files/resources/publications/annual-reports/esg_ar2020_21.pdf

[7] AWS, Price List files for Lambda, API Gateway, DynamoDB, S3, Amplify, Cognito, CloudWatch and data transfer, ap-southeast-1. https://aws.amazon.com/pricing/

[8] AWS, “Amazon Location Service pricing” and Routes pricing (Core bucket), ap-southeast-2. https://aws.amazon.com/location/pricing/

[9] US Federal Reserve, “H.10 Foreign Exchange Rates” (30 Oct 2026). https://www.federalreserve.gov/releases/h10/current/

[10] Dell Singapore, “PowerEdge R350.” https://www.dell.com/en-sg/shop/dell-poweredge-servers/poweredge-r350/spd/poweredge-r350/asper350_vi_vp

[11] IT Block, “Single server colocation Singapore.” https://www.itblock.sg/single-server-colocation-singapore

[12] SPTel, “Business broadband.” https://sptel.com/promotion/business-broadband-singapore/

[13] SP Group, “Tariff information.” https://www.spgroup.com.sg/our-services/utilities/tariff-information

[14] Indeed Singapore, “Systems administrator salaries.” https://sg.indeed.com/career/systems-administrator/salaries

[15] AWS, “DynamoDB resilience and replication.” https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Replication.html

[16] PDPC, “Data protection obligations.” https://www.pdpc.gov.sg/overview-of-pdpa/the-legislation/personal-data-protection-act/data-protection-obligations

[17] M. Held and R. M. Karp, “A dynamic programming approach to sequencing problems,” *J. SIAM*, 10(1), 1962.

[18] J. Brooke, “SUS: a quick and dirty usability scale,” in *Usability Evaluation in Industry*, 1996.

[19] A. Bangor, P. Kortum, J. Miller, “An empirical evaluation of the System Usability Scale,” *IJHCI*, 24(6), 2008.

All web sources accessed 4 November 2026.

Draft notes (not part of the report)

- **Page limit:** build `report/final.tex` (notes hidden) and keep it to 10 pages including cover and references.
- **Checked against:** `main` at `6a26188` (all PRs merged, deployed 5 Nov); test counts (340) and the `/api/plan` throttle (5/s, burst 10) re-checked there.
- **Cover:** names and group number only; the professor said the team number is enough, and Canvas has the IDs.
- **Chien's original:** kept unchanged on branch `docs/final-report-draft`, with his `materials.tex` and `planning_notes.te` this draft is on `docs/final-report-v1`.
- **Older drafts:** `Final.docx`, the slides and Lian's backend reference describe an earlier version (camera-based traffic, backend greedy optimiser, SNS alarms, Python 3.11). This draft follows the final code; update the slides to match.
- **Sources for every number:** the references, the code on `main`, and `frontend/eval/results/` for Section 4.2.
- **Spec checklist:** (1) business case ✓ (2) business model + on-prem comparison ✓ (3) architecture, algorithm, requirements, user journey ✓ (4) system evaluation: route quality done (frontend/eval); deployment checks and live timings done on 5 Nov (grid-freshness numbers pending); user study still needs results (5) conclusion ✓ (6) code explanation + public URL: make repo public; assumptions stated ✓; diagrams ✓; AI declaration: each member.